Topological Sort

SHSID CPC

Implementation (BFS)

For each time step, visit the “oldest” node with in-degree 0 (i.e. there are no edges pointing towards that node), delete it, and add all the new nodes with in-degree 0 into the BFS queue

Implementation (BFS)

```cpp
#include <bits/stdc++.h>
using namespace std;
int n, m;
int main() {
    cin >> n >> m;
    vector<int> graph[n + 1];

    Adjacency matrix: "vector graph[n+1]" is an array of vectors, and the x'th vector contains all the
    nodes x can go to via one edge. Here, the length is set to n + 1 because we need it to contain the
    indices 1 to n (whereas an array with length n only goes to n - 1).*

    vector<int> in_degree( n: n + 1);
    for (int i = 1; i <= m; i++) {
        int a, b;
        cin >> a >> b;
        graph[a].push_back(b);
        in_degree[b]++;

        For every new edge, its target node's (in this case, b's) in-degree increases by 1.

    }
    queue<int> queue;
    for (int i = 1; i <= n; i++) {
        if (in_degree[i] == 0) {
            queue.push( x: i);

            Initialization of the BFS queue. We find all nodes that initially have an in-degree of 0 (source
            nodes) and place them in our queue.

        }
    }
```

```cpp
    vector<int> top_sort = {0};
    while (!queue.empty()) {
        int curr = queue.front();

            The front of the queue is the oldest node that entered the queue (this should be apparent).

        queue.pop();
        top_sort.push_back(curr);

            Since we know for certain that curr has in-degree 0, we can place it in our topological sort list.

        for (int next: graph[curr]) {
            in_degree[next]--;

                Node "next" can be directly by curr via an edge. Since we have deleted curr from the
                graph, the in-degree of "next" -= 1.

            if (in_degree[next] == 0) {
                queue.push( x: next);
            }
        }
    }
    if (top_sort.size() == n) {
        for (int i = 1; i < n; i++) cout << top_sort[i] << ' ';
        cout << top_sort.back();
    }
    else cout << "IMPOSSIBLE";
    return 0;
}
```

Implementation (DFS)

For each time step, visit the “newest” node with in-degree 0. Mostly similar to the BFS, but a bit more annoying to implement and understand and absolutely useless in 99% of all cases

CSES Problem Set

Game Routes

TASK | SUBMIT | RESULTS | ANALYSIS | STATISTICS | TESTS | QUEUE

Time limit: 1.00 s **Memory limit:** 512 MB

A game has n levels, connected by m teleporters, and your task is to get from level 1 to level n. The game has been designed so that there are no directed cycles in the underlying graph. In how many ways can you complete the game?

Input

The first input line has two integers n and m: the number of levels and teleporters. The levels are numbered $1, 2, \ldots, n$.

After this, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from level a to level b.

Output

Print one integer: the number of ways you can complete the game. Since the result may be large, print it modulo $10^9 + 7$.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Example

Input:

```
4 5
1 2
2 4
1 3
3 4
1 4
```

Output:

```
3
```

Game Routes (CSES Problem Set)

Observation: the number of ways to reach a certain level is the sum of the number of ways to get to the levels that can reach the current level via an edge.

Trivial DP with a transition function looping through all the edges that have the current level as a target node. Do a topological sort on the levels (it is ensured that the graph is a DAG, or Directed Acyclic Graph, so a topological sort is possible) to ensure that the ancestor nodes are accessed and have a calculated DP value before accessing the child nodes.

Game Routes (CSES Problem Set)

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long
const int N = 100005, mod = 1000000007;
vector<int> edges[N], top_sort = {0}, back[N];
int n, m, in_degree[N], a, b, dp[N];
queue<int> q;
signed main() {
    cin >> n >> m;
    for (int i = 1; i <= m; i++) {
        cin >> a >> b;
        if (b == 1)continue;
        edges[a].push_back(b);
        in_degree[b]++;
        back[b].push_back(a);
    }
    for (int i = 1; i <= n; i++) {
        if (in_degree[i] == 0) {
            q.push( x: i);
        }
    }
```

```cpp
    while (!q.empty()) {
        int curr = q.front();
        q.pop();
        top_sort.push_back(curr);
        for (int next: edges[curr]) {
            if (!--in_degree[next]) {
                q.push( x: next);
            }
        }
    }
    dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        for (auto x : long long : back[top_sort[i]]) {
            dp[top_sort[i]] += dp[x];
            dp[top_sort[i]] %= mod;
        }
    }
    cout << dp[n];
    return 0;
}
```

USACO 2020 February Contest, Gold
Problem 1. Timeline

Return to Problem List

Contest has ended.

Analysis mode

English (en)

Bessie attended N milking sessions ($1 \le N \le 10^5$) over the past M days ($2 \le M \le 10^9$). However, she is having trouble remembering when she attended each session.

For each session $i = 1 \ldots N$, she knows that it occurred no earlier than day S_i ($1 \le S_i \le M$). Additionally, Bessie has C memories ($1 \le C \le 10^5$), each described by a triple (a, b, x), where she recalls that session b happened at least x days after a.

Help Bessie by computing the earliest possible date of occurrence for each milking session. It is guaranteed that Bessie did not remember incorrectly; in other words, there exists an assignment of sessions to days in the range $1 \ldots M$ such that all constraints from her memories are satisfied.

SCORING:

- Test cases 2-4 satisfy $N, C \le 10^3$.
- Test cases 5-10 satisfy no additional constraints.

INPUT FORMAT (file timeline.in):

The first line of input contains N, M, and C.

The next line contains N space-separated integers $S_1, S_2, \ldots, S_N$. Each is in the range $1 \ldots M$.

The next C lines contain three integers, a, b, and x indicating that session b happened at least x days after a. For each line, $a \ne b$, a and b are in the range $1 \ldots N$, and x is in the range $1 \ldots M$.

OUTPUT FORMAT (file timeline.out):

Output N lines giving the earliest possible date of occurrence for each session.

SAMPLE INPUT:

```
4 10 3
1 2 3 4
1 2 5
2 4 2
3 4 4
```

SAMPLE OUTPUT:

```
1
6
3
8
```

Session two occurred at least five days after session one, so it cannot have occurred before day $1 + 5 = 6$. Session four occurred at least two days after session two, so it cannot have occurred before day $6 + 2 = 8$.

Problem credits: Mark Gordon

USACO 2020 Feb Gold P1. Timeline

We can turn this problem into a graph problem, where each memory can be turned into a directed edge from the earlier session to the later session with edge weight x, and each condition S_i as an edge from time 0 to session i. Since Bessie's memory is not flawed and all x (edge weights) are positive, the graph must be a DAG, and we can thus do some fun topological sort.

Idea: The minimum time of a session is dictated by the latest (most strict) constraint. This makes us think of the BFS implementation of topological sort, where a node turns into a source only when all of its in-edges are deleted and nodes are processed in a chronological order. A slightly modified topological source will thus pass this problem.

USACO 2020 Feb Gold P1. Timeline

```cpp
#include "bits/stdc++.h"
using namespace std;
const int N = 100005;
int n,m,c,s[N],in[N],a,b,x;
vector<pair<int,int>> adj[N];
queue<int> q;
int main() {
    freopen("timeline.in","r",stdin);
    freopen("timeline.out","w",stdout);
    cin >> n >> m >> c;
    for (int i = 1; i <= n; i++) {
        cin >> s[i];
    }
    for (int i = 0; i < c; i++) {
        cin >> a >> b >> x;
        adj[a].emplace_back( a: b, b: x);
        in[b]++;
    }
    for (int i = 1; i <= n; i++) {
        if (!in[i]) {
            q.push( x: i);
        }
    }
    while (!q.empty()) {
        x = q.front();
        q.pop();
        for(auto& t : pair<int, int> & : adj[x]) {
            s[t.first] = max(s[t.first],s[x]+t.second);
            if (!--in[t.first]) q.push( x: t.first);
        }
    }
    for (int i = 1; i <= n; i++) {
        cout << s[i] << '\n';
    }
    return 0;
}
```

Codeforces Round 656 (Div. 3) E. Directing Edges

E. Directing Edges

time limit per test: 3 seconds

memory limit per test: 256 megabytes

You are given a graph consisting of n vertices and m edges. It is not guaranteed that the given graph is connected. Some edges are already directed and you can't change their direction. Other edges are undirected and you have to choose some direction for all these edges.

You have to direct undirected edges in such a way that the resulting graph is directed and acyclic (i.e. the graph with all edges directed and having no directed cycles). Note that you have to direct **all** undirected edges.

You have to answer t independent test cases.

Input

The first line of the input contains one integer t ($1 \le t \le 2 \cdot 10^4$) — the number of test cases. Then t test cases follow.

The first line of the test case contains two integers n and m ($2 \le n \le 2 \cdot 10^5$, $1 \le m \le min(2 \cdot 10^5, \frac{n(n-1)}{2})$) — the number of vertices and the number of edges in the graph, respectively.

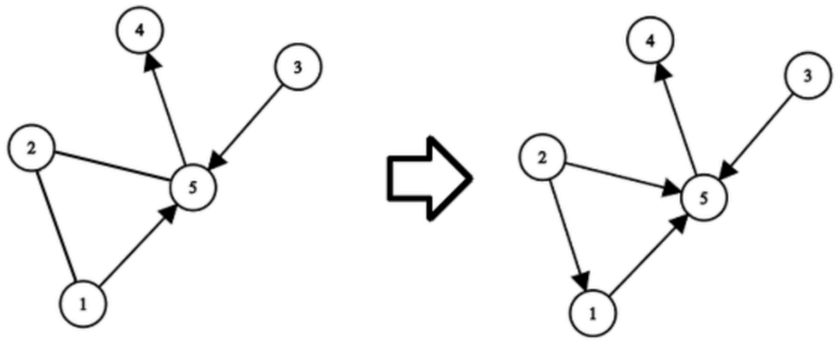
The next m lines describe edges of the graph. The i-th edge is described with three integers t_i, x_i and y_i ($t_i \in [0; 1]$, $1 \le x_i, y_i \le n$) — the type of the edge ($t_i = 0$ if the edge is undirected and $t_i = 1$ if the edge is directed) and vertices this edge connects (the undirected edge connects vertices x_i and y_i and directed edge is going from the vertex x_i to the vertex y_i). It is guaranteed that the graph do not contain self-loops (i.e. edges from the vertex to itself) and multiple edges (i.e. for each pair (x_i, y_i) there are no other pairs (x_i, y_i) or (y_i, x_i)).

It is guaranteed that both sum n and sum m do not exceed $2 \cdot 10^5$ ($\sum n \le 2 \cdot 10^5$; $\sum m \le 2 \cdot 10^5$).

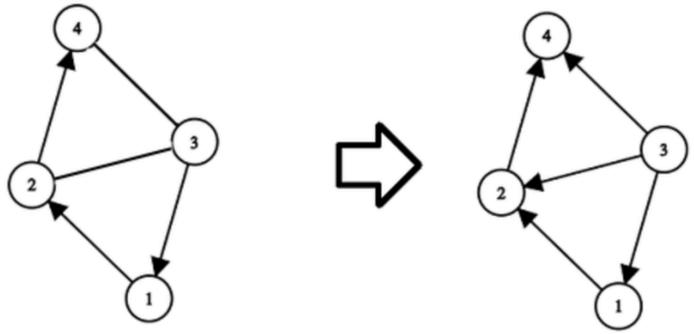
Output

For each test case print the answer — "NO" if it is impossible to direct undirected edges in such a way that the resulting graph is directed and acyclic, otherwise print "YES" on the first line and m lines describing edges of the resulted directed acyclic graph (**in any order**). Note that you cannot change the direction of the already directed edges. If there are several answers, you can print any.

Explanation of the second test case of the example:



Explanation of the third test case of the example:



Codeforces Round 656 (Div. 3) E. Directing Edges

Notice that if the directed edges do not form a cycle, there must be a way to direct the undirected edges such that no cycle is formed.

Thus, we can do a topological sort with the directed edges to check for cycles. If there is no cycle, we can use the results from the sort to position the undirected edges (e.g. if some edge is between a and b, and b is earlier in the topological sort than a, then we can direct the edge from b to a).

Codeforces Round 656 (Div. 3) E. Directing Edges

```cpp
#include <bits/stdc++.h>
using namespace std;
int n,m,t,a,b;
bool c;
int main() {
    cin >> t;
    while(t--) {
        cin >> n >> m;
        vector<int> graph[n + 1],in_degree( n: n + 1),top_sort = {0},order,udir[n + 1];
        order.resize( new_size: n + 1);
        for (int i = 1; i <= m; i++) {
            cin >> c >> a >> b;
            if(c) {
                graph[a].push_back(b);
                in_degree[b]++;
            }
            else udir[a].push_back(b);
        }
        queue<int> q;
        for (int i = 1; i <= n; i++) {
            if (in_degree[i] == 0) {
                q.push( x: i);
            }
        }
```

```cpp
        while (!q.empty()) {
            int curr = q.front();
            q.pop();
            top_sort.push_back(curr);
            order[curr] = top_sort.size() - 1;
            for (int next: graph[curr]) {
                in_degree[next]--;
                if (in_degree[next] == 0) {
                    q.push( x: next);
                }
            }
        }
        if (top_sort.size() == n + 1) {
            cout << "YES\n";
            for(int i = 1; i <= n; i++){
                for(auto j :int : graph[i])cout << i << ' ' << j << '\n';
                for(auto j :int : udir[i]){
                    if(order[i] < order[j])cout << i << ' ' << j << '\n';
                    else cout << j << ' ' << i << '\n';
                }
            }
        }
        else cout << "NO\n";
    }
    return 0;
}
```

USACO 2018 US Open Contest, Gold
Problem 2. Milking Order

Return to Problem List

Contest has ended.

Log in to allow submissions in analysis mode

English (en)

Farmer John's N cows ($1 \le N \le 10^5$), numbered $1 \ldots N$ as always, happen to have too much time on their hooves. As a result, they have worked out a complex social hierarchy related to the order in which Farmer John milks them every morning.

After weeks of study, Farmer John has made M observations about his cows' social structure ($1 \le M \le 50,000$). Each observation is an ordered list of some of his cows, indicating that these cows should be milked in the same order in which they appear in this list. For example, if one of Farmer John's observations is the list 2, 5, 1, Farmer John should milk cow 2 sometime before he milks cow 5, who should be milked sometime before he milks cow 1.

Farmer John's observations are prioritized, so his goal is to maximize the value of X for which his milking order meets the conditions outlined in the first X observations. If multiple milking orders satisfy these first X conditions, Farmer John believes that it is a longstanding tradition that cows with lower numbers outrank those with higher numbers, so he would like to milk the lowest-numbered cows first. More formally, if multiple milking orders satisfy these conditions, Farmer John would like to use the lexicographically smallest one. An ordering x is lexicographically smaller than an ordering y if for some j, $x_i = y_i$ for all $i < j$ and $x_j < y_j$ (in other words, the two orderings are identical up to a certain point, at which x is smaller than y).

Please help Farmer John determine the best order in which to milk his cows.

INPUT FORMAT (file milkorder.in):

The first line contains N and M. The next M lines each describe an observation. Line $i + 1$ describes observation i, and starts with the number of cows m_i listed in the observation followed by the list of m_i integers giving the ordering of cows in the observation. The sum of the m_i's is at most $200,000$.

OUTPUT FORMAT (file milkorder.out):

Output N space-separated integers, giving a permutation of $1 \ldots N$ containing the order in which Farmer John should milk his cows.

SAMPLE INPUT:

```
4 3
3 1 2 3
2 4 2
3 3 4 1
```

SAMPLE OUTPUT:

```
1 4 2 3
```

Here, Farmer John has four cows and should milk cow 1 before cow 2 and cow 2 before cow 3 (the first observation), cow 4 before cow 2 (the second observation), and cow 3 before cow 4 and cow 4 before cow 1 (the third observation). The first two observations can be satisfied simultaneously, but Farmer John cannot meet all of these criteria at once, as to do so would require that cow 1 come before cow 3 and cow 3 before cow 1.

This means there are two possible orderings: 1 4 2 3 and 4 1 2 3, the first being lexicographically smaller.

Problem credits: Jay Leeds

Milking Order

Binary search on X, and check if it is possible to complete a topological sort.
Since number of x before y is within 2e5, the complexity for this is E log M.

After finding X, do a final topological sort with a priority queue of indices to fulfill the lexicographically minimum condition

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<pair<int,int>> layer[50005];
vector<int> G[100005];
int in[100005];
int grandAns[100005];

bool check(int upp){
    for (int i = 1; i <= n; i++){
        G[i].clear();
        in[i] = 0;
    }
    for (int i = 1; i <= upp; i++){
        for (auto [u, v] : layer[i]){
            G[u].push_back(v);
            in[v]++;
        }
    }
    queue<int> q;
    for (int i = 1; i <= n; i++){
        if (in[i] == 0) q.push(i);
    }
    int cnt = 0;
    while (!q.empty()){
        int x = q.front(); q.pop();
        cnt++;
        for (int v : G[x]){
            in[v]--;
            if (in[v]==0)
            q.push(v);
        }
    }
    return cnt == n;
}
```

```cpp
void optimize(int upp){
    for (int i = 1; i <= n; i++){
        G[i].clear();
        in[i] = 0;
    }
    for (int i = 1; i <= upp; i++){
        for (auto [u, v] : layer[i]){
            G[u].push_back(v);
            in[v]++;
        }
    }
    // ascendingly sorted
    priority_queue<int, vector<int>, greater<int>> q;
    for (int i = 1; i <= n; i++){
        if (in[i] == 0) q.push(i);
    }
    int cnt = 0;
    while (!q.empty()){
        int x = q.top(); q.pop();
        grandAns[++cnt] = x;
        for (int v : G[x]){
            in[v]--;
            if (in[v]==0)
            q.push(v);
        }
    }
}

int main(){
    cin >> n >> m;
    for (int i = 1; i <= m; i++){
        int mi, prev; cin >> mi >> prev;
        for (int j = 2; j <= mi; j++){
            int curr; cin >> curr;
            layer[i].push_back({prev, curr});
            prev = curr;
        }
    }
    int l = 1, r = m, ans = 0;
    while (l <= r){
        int mid = (l+r)>>1;
        if (check(mid)) {ans = mid; l = mid+1;}
        else r = mid-1;
    }
    // cout << ans <<'\n';
    optimize(ans);
    for (int i = 1; i <= n; i++){
        cout << grandAns[i] << ' ';
    }
    return 0;
}
```

CITY PLANNING

A space station is being built on a planet. The station will employ N persons. They have to live somewhere, and for that, a city will be built around the station. The land surrounding the station is divided into equal-sized square lots, and an apartment building with up to K floors may be built on each lot. Each building shall have exactly one apartment on each floor, and each person shall live in a separate apartment. The lots are assigned coordinates in the form (x, y), where the space station has coordinates (0, 0) and the rest of the lots are numbered as shown below:

…	…	…	…	…
…	(-1, 1)	(0, 1)	(1, 1)	…
…	(-1, 0)	(0, 0)	(1, 0)	…
…	(-1, -1)	(0, -1)	(1, -1)	…
…	…	…	…	…

As the traffic can only flow on streets between the lots, the distance of the lot (x, y) from the space station is $|x| + |y| - 1$ units.

The cost of building a house is equal to the sum of the costs of each floor. It is known that the cost to build a floor depends on the height of the floor, but not on the location of the building.

The houses to be built will last for 30 years. People living in those houses will go to work in the space station, and to transport them to and from the station during these 30 years will cost $T \cdot d$ per person, where d is the distance of the person's building from the station.

We can assume that the planet is big enough and the city to be built occupies a small enough portion of the surface that we do not need to consider the curvature of the planet's surface.

Your task is to write a program that determines the minimal total cost of building the houses and operating the transportation system for the 30 years.